Performance Analysis

1. Serialization

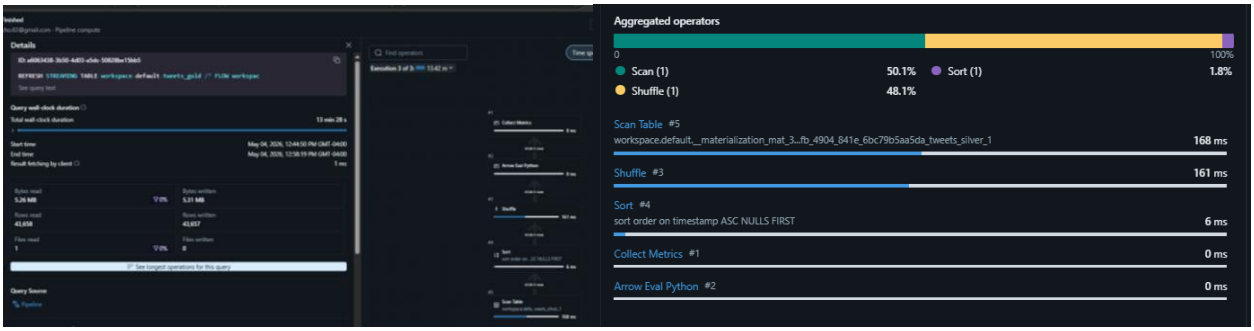
One performance consideration in the Gold transformation pipeline is serialization overhead introduced during the sentiment prediction step. This happens because the pipeline uses a Python-based MLflow Spark UDF to run model inference on tweet text instead of relying entirely on native Spark execution.

In the Gold transformation, sentiment predictions are generated here, which is then applied to every incoming record in the Gold transformation:

Since the Gold pipeline reads from the tweets_silver streaming table and applies this model to every tweet, each record must pass through Python model inference before continuing through the rest of the transformation. The pipeline does attempt to optimize this by enabling Apache Arrow and increasing batch sizes. This helps reduce the overhead of moving data between Spark and Python by processing records in batches instead of individually, which is appropriate for this type of inference workload. However, Python execution time in the operator breakdown appears minimal relative to the total query runtime.

Spark UI confirms that Python-based execution is occurring through the presence of the Arrow Eval Python operator in the Gold transformation execution plan. However, the measured performance metrics suggest that serialization overhead is not the dominant bottleneck in this workload. The Gold transformation processed 43,658 rows, read 5.26 MB, wrote 5.31 MB, and completed in approximately 13.5 minutes, while showing no spill to disk. The operator breakdown indicates that the majority of execution time was spent in Scan (50.1%) and Shuffle (48.1%), with Python execution contributing minimally. This suggests that while the MLflow Python UDF introduces serialization overhead, the more significant runtime cost likely comes from data movement rather than model inference execution itself.

A potential optimization would be replacing the Python-based MLflow Spark UDF with a Spark-native inference approach, if supported, to avoid repeated data transfer between Spark and Python runtimes. Another option would be increasing the Arrow batch size beyond the current setting of 1000 records per batch to reduce the number of cross-runtime serialization events. However, based on the Spark UI metrics, optimizing shuffle and data movement would likely provide a greater overall performance improvement for this specific workload.



2. Shuffle

Another major performance consideration in the Gold transformation pipeline is shuffle overhead caused by repartitioning the incoming streaming dataset before sentiment inference. This operation forces Spark to redistribute records across executors, creating shuffle overhead through network communication, task coordination, and intermediate data movement. However, in this case, repartitioning was introduced intentionally as a performance optimization. Before adding .repartition(200), the Gold transformation was taking over 8 hours to complete. After introducing repartitioning, runtime dropped to approximately 13.5 minutes, showing that increasing parallelism significantly improved throughput for the ML inference workload.

Spark UI metrics show the Gold transformation processed 43,658 rows, with 5.26 MB read and 5.31 MB written, while shuffle accounted for 48.1% of total execution time, nearly equal to scan operations at 50.1%. This indicates that while repartitioning improved performance dramatically, shuffle remains a significant runtime cost. A logical next optimization would be tuning the partition count rather than assuming 200 is optimal. Testing smaller partition counts such as .repartition(100) or .repartition(150) could reduce shuffle overhead while still maintaining enough parallelism for efficient model inference, potentially improving runtime further.

3. Spill

Another performance consideration in the Gold transformation pipeline is shuffle spill, which occurs when Spark writes intermediate shuffle data to disk due to insufficient executor memory. Spill can significantly slow performance because disk I/O is much slower than in-memory processing.

In this workload, Spark UI shows 0 bytes spilled to disk, indicating that the current workload fits within available executor memory despite the explicit repartitioning step. Since shuffle already accounts for 48.1% of total runtime, any spill would likely have increased execution time substantially beyond the observed 13.5 minutes. The absence of spill suggests that the current partitioning strategy prevents memory pressure effectively.

However, because the workload processed only 43,658 rows, the partition count may still be higher than necessary. Testing smaller partition counts such as .repartition(100) or .repartition(150) could reduce shuffle overhead while maintaining in-memory execution.